Taller 3 — Flappy Bird: el pájaro cobra vida

Nombre: Samuel Alejandro Alzate Molina

Promoción: Ingeniería de Software 18A

Sesión 3 · Corte 1 · **Individual** · Versión para los equipos de **Flappy Bird**

Entrega: al terminar la clase, un documento por estudiante (PDF o enlace) en su fila del Excel del corte

> **Qué van a hacer.** Es el mismo taller que el de Pac-Man, pero sobre el clásico de su equipo. El pájaro ya vuela: cae con gravedad y sube con cada aleteo. Ahora tiene que **verse** y **oírse** como el de verdad: batir las alas solo cuando sube, inclinarse según cómo cae, hacer "fla" con cada aleteo y morirse al tocar el piso. La pregunta de fondo de todo el taller: **¿con qué dato se decide lo que se ve?**
>
> **Es individual.** Pueden discutir ideas con sus compañeros, pero el código y el documento son de cada uno.
>
> **Cómo se trabaja.** Cada nivel empieza con **Antes de programar**: respondan en papel y copien esas respuestas al documento **antes** de escribir código. Después comparen su predicción con lo que ven y lo que imprime la consola (`print`). Una predicción equivocada pero bien explicada vale más que una correcta sin explicación.
>
> **Cómo usar las ayudas.** Cada nivel trae: **Qué ya saben** (lo de la demo que se usa ahí), una **Guía** por pregunta con un ejemplo **con otros números o con otra situación**, un **Esqueleto** del código y **Cómo saber que funciona**. En el esqueleto, cada _____es algo que tienen que escribir ustedes. Traigan **audífonos**: el nivel 2 se comprueba escuchando.

Duración

	Grupos de 3 horas	Grupos de 2 horas
Tiempo para el taller	120 minutos	60 minutos
Hasta dónde se espera llegar	Nivel 3 completo	Nivel 2 (el nivel 3 da acceso al 5.0)

Preparación (5 minutos)

1. Descarguen **Base Taller 3 - Flappy Bird.zip** de Teams y descomprímanlo.
2. En Godot: **Import** → elijan el *project.godot* de la carpeta → **Import & Edit**.
3. F5. El pájaro avanza solo hacia la derecha, cae y sube con **Espacio** (o clic). Ya tiene un **AnimatedSprite2D** con las animaciones "planear" y "aletear", pero todavía no las usa.

4. Abran *jugador.gd*. Arriba están `gravedad` (980 px/s²), `impulso` (350 px/s) y `@onready var sprite`. En `_physics_process` hay un comentario que dice dónde va lo del taller. El vuelo está en `mover()`: no

hace falta tocarla.

Si su equipo ya tiene su propio proyecto de Flappy Bird, pueden trabajar en él: copien la carpeta *assets* de la base y armen el AnimatedSprite2D como en la demo.

Nivel 1 — Las alas baten solo cuando sube

Qué ya saben. En la demo, `update_animation(direction)` elegía la animación con las teclas: con alguna tecla presionada, "comer", y si no, "quieto".

Antes de programar

1. Si eligen la animación con `Input.is_action_just_pressed("aletear")`, ¿cuánto tiempo se ve "aletear" después de cada aleteo? (`just_pressed` es verdadero en un solo frame.)

Rta = Solo dura un frame porque is_action_just_pressed detecta la tecla presionada en un solo intstante

2. Después de un aleteo, la velocidad vertical arranca en −350 px/s y la gravedad le suma 980 px/s cada segundo. ¿Cuántos segundos sigue subiendo? ¿Cuántos frames son a 60 por segundo?

Rta = Sigue subiendo unos 0.35 segundos pporque se supone que dividiendo los 350 de impulso entre los 980 de gravedad y pues despues multiplicando eso por 60 fps da como unos 21 o 22 frames

3. Cuando el pájaro está apoyado en el piso, ¿cuánto vale `get_real_velocity().y`? ¿Qué animación debería verse?

Rta = En el piso vale 0 y lo que se veria es la animación planear

Guía para la 1. Un frame a 60 FPS dura 1/60 de segundo. Con otra situación: un flash de cámara que dura 16 milésimas, ¿alcanzan a ver de qué color era?

Guía para la 2. Con otros números: una pelota lanzada hacia arriba a 20 m/s, con una gravedad de 10 m/s², deja de subir cuando su velocidad llega a 0, a los 20 ÷ 10 = 2 s. Para pasarlo a frames, multipliquen por 60.

Guía para la 3. El piso frena al pájaro. Pongan `print(get_real_velocity())` y déjenlo caer.

Programar

1. Escriban `actualizar_animacion()`: si la velocidad **real** en y es menor que −1 (sube), "aletear"; si no, "planear".
2. Llámenla en `_physics_process`, después de `mover()`.
3. Cuenten cuántos frames dura "aletear" después de un aleteo (pueden imprimir `sprite.animation` en cada frame) y compárenlo con la predicción 2. Captura.

Esqueleto.

```
func actualizar_animacion():
    var vy = get_real_velocity().____
    if vy < ____:                  # sube
        sprite.play(____)
    else:
        sprite.play(____)
```

Por qué −1 y no 0. Los decimales del computador casi nunca dan cero exacto. Prueben `print(0.1 + 0.2 ==`

`0.3)`: sale `false`. Un umbral pequeño evita que la animación parpadee justo en el punto más alto.

Cómo saber que funciona. Con cada aleteo, las alas baten durante un poco más de un tercio de segundo y luego el pájaro planea. Apoyado en el piso, planea. Con un `print` por frame deberían contar **unos 22 frames** de "aletear" por aleteo. Si las alas solo se mueven mientras mantienen la tecla, están usando la tecla y no la velocidad.

Mi script de Jugador.gd quedaria asi:

Llamo a la funcion en _physics_process

```
func _physics_process(delta):

mover(delta)

actualizar_animacion()
```

Y asi quedaria la nueva funcion:

```
func actualizar_animacion():

var vy = get_real_velocity().y

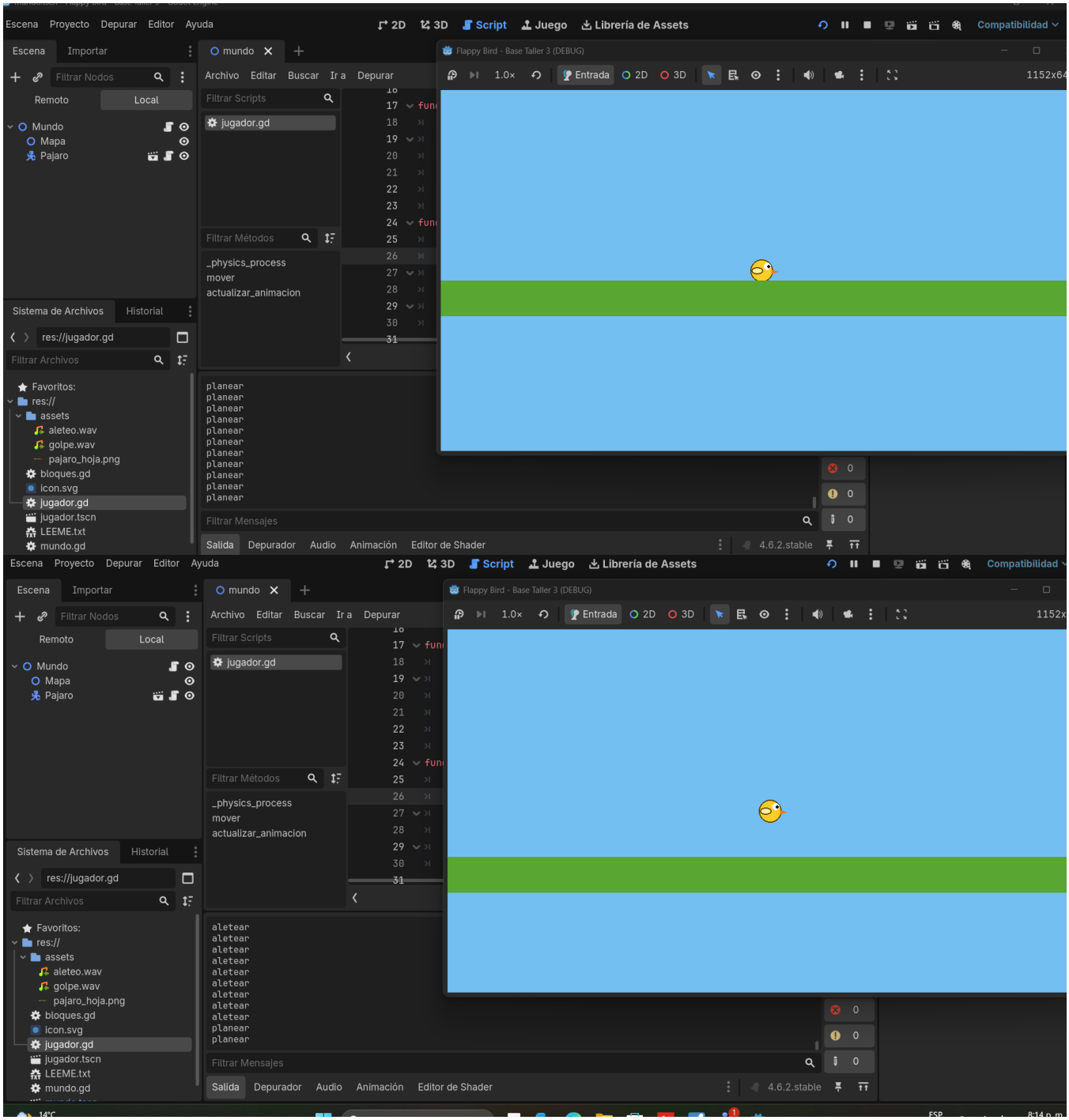
        print(sprite.animation)

if vy < -1:                # sube

sprite.play("aletear")

else:

sprite.play("planear")
```



Mi predicción sí le dio porque el aleteo duró más o menos 22 frames y es justo el tiempo que se gasta la gravedad en frenar el impulso hacia arriba

Nivel 2 — Inclinarse según cómo cae, y el aleteo

Qué ya saben. En Godot, `rotation` está en **radianes** y `deg_to_rad()` pasa de grados a radianes. Una rotación positiva gira en el sentido de las manecillas del reloj, porque en la pantalla el eje Y crece hacia abajo. `clampf(x, minimo, maximo)` deja a `x` dentro de un rango.

Antes de programar

1. Usen la inclinación `grados = clampf(vy * 0.15, -25, 90)`. ¿Cuántos grados da justo después de un aleteo (vy = −350)? ¿Y en el punto más alto (vy = 0)?

Rta = Al aletear dariia como -52 pero como el clampf no lo deja bajar de –25 pues se queda en -25 y arriba en el punto más alto da 0

2. ¿Con qué velocidad de caída queda el pájaro mirando hacia abajo (90°)? ¿Cuántos frames después de un aleteo llega a esa velocidad, si no vuelve a aletear?

Rta = Queda mirando recto para abajo a los 600 de velocidad y se tarda como unos 37 frames en llegar ahí desde que empieza a caer

3. El sonido *aleteo.wav* dura 0.25 s. En Pac-Man, el waka **no** se reiniciaba (se preguntaba por `playing`). Si aquí hacen lo mismo y el jugador aletea dos veces con 0.1 s de diferencia, ¿cuántos "fla" se oyen? ¿Cuántos deberían oírse?

Rta = Que si hundo la tecla rápido el juego se come los sonidos porque espera a que termine el anterior tienen que oírse todos los aleteos.

Guía para la 1. Con otros números: con un factor de 0.2 y vy = −100, da −20°; si el mínimo fuera −15, `clampf` lo deja en −15. Con positivos: el pájaro mira hacia abajo, porque Y crece hacia abajo.

Guía para la 2. Despejen vy en vy · 0.15 = 90. Después, con otros números: si la velocidad sube 10 cada frame y arranca en −50, para llegar a 100 necesita (100 − (−50)) ÷ 10 = 15 frames. Aquí cada frame suma 980 ÷ 60.

Guía para la 3. ¿Cuál es la diferencia entre un sonido que **acompaña un estado** (Pac-Man avanzando) y uno que **marca un evento** (cada aleteo)?

Programar

1. Escriban `inclinar()`: calculen los grados con la fórmula de la pregunta 1 a partir de la velocidad real y pónganlos en `sprite.rotation`. Giren solo el sprite: la colisión no necesita girar.
2. Agreguen al pájaro un **AudioStreamPlayer2D** llamado **Aleteo**, con Stream = *assets/aleteo.wav*, y su `@onready var sonido_aleteo = $Aleteo`.
3. En `_physics_process`, cuando se presione "aletear", reproduzcan el sonido **en cada aleteo**, aunque el anterior no haya terminado.
4. Con audífonos, prueben un momento la versión que pregunta `if not sonido_aleteo.playing` y aleteen rápido. Describan la diferencia.

Esqueleto.

```
func inclinar():
    var grados = clampf(get_real_velocity().y * ____, ____, ____)
    sprite.rotation = ____(grados)

# en _physics_process, después de mover():
    if Input.is_action_just_pressed(____):
        sonido_aleteo.____()
```

Cómo saber que funciona. Al aletear, el pico apunta hacia arriba (−25°); al caer, va bajando hasta apuntar al piso. Con `print(rad_to_deg(sprite.rotation))`: −25 justo después del aleteo y 90 **unos 59 frames (casi un segundo)** después, si no vuelven a aletear. Aleteando rápido, cada aleteo suena. Con el `if not ...playing`, se pierden aleteos.

Como quedo mi código:

Llame la nueva funcion osea la de inclinar:

func _physics_process(delta):

mover(delta)

actualizar_animacion()

inclinar()

Puse el sonido

if Input.is_action_just_pressed("aletear"):

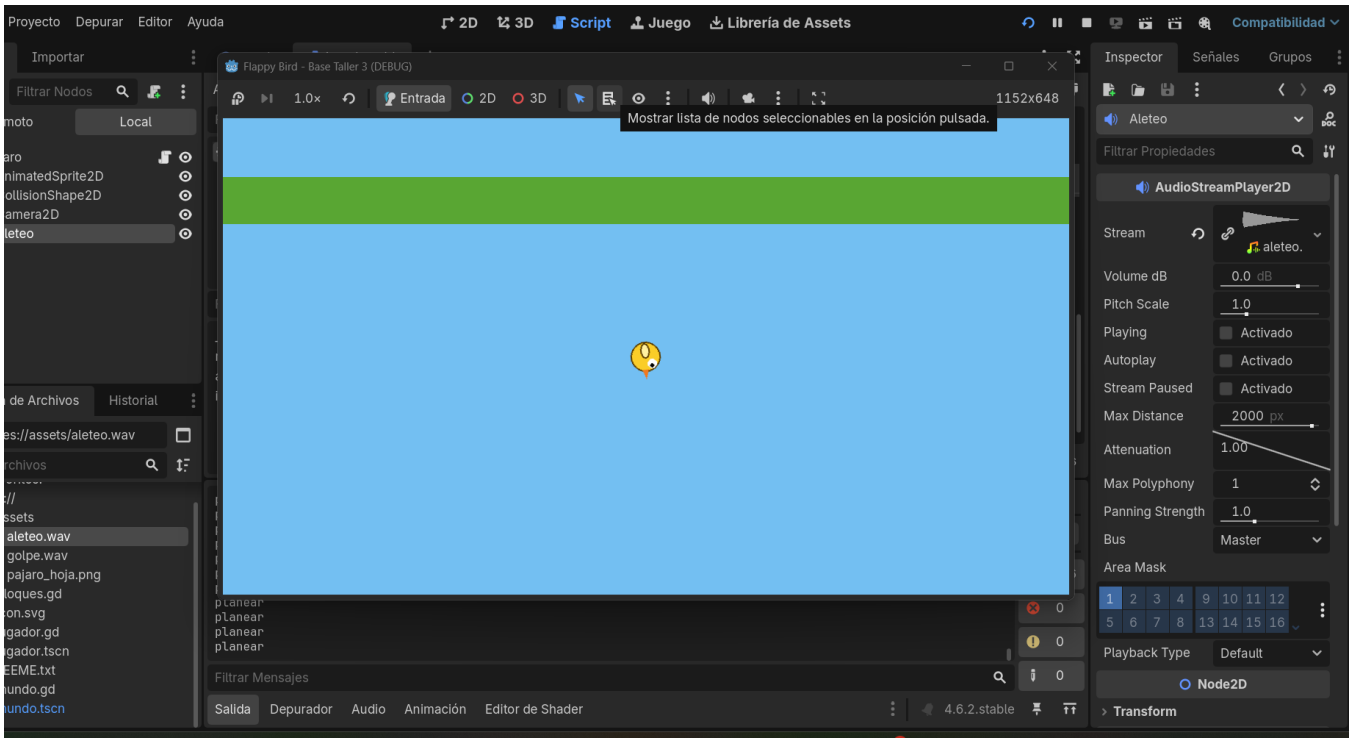
sonido_aleteo.play()

Y la funcion de inclinar

func inclinar():

var grados = clampf(get_real_velocity().y * 0.15, -25, 90)

sprite.rotation = deg_to_rad(grados)



Ya hace el sonido cuando le doy al espacio y mi predicción si quedo bien porque el pájaro arrancó inclinado a –25 grados al saltar y fue rotando hacia abajo mientras cae

Nivel 3 — El ritmo de las alas, y morir

Qué ya saben. En el panel SpriteFrames cada animación tiene sus FPS y su **Loop**. Y `await` hace que una función espere algo sin congelar el juego.

Antes de programar

1. La propiedad `speed_scale` multiplica los FPS de la animación. Queremos que las alas baten al doble justo después del aleteo y a ritmo normal en el punto más alto. Escriban una fórmula de `speed_scale` a partir de `vy` y de `impulso` (350). ¿Cuánto da 9 frames después del aleteo, cuando vy ≈ −203?

Rta = La fórmula que sirve es 1 + -vy / 350.0 en los 9 frames de haber aleteado el pájaro va a una velocidad de unos –203 entonces haciendo la cuenta la velocidad da más o menos 1.58

2. La animación "morir" va a tener 6 cuadros a 10 FPS. ¿Cuánto dura en segundos? ¿Y en frames de física?

Rta = En segundos dura 0.6 segundos porque se divide 6 entre 10 y pasándolo a frames multiplicando por 60 da 36 frames

3. Si llaman `morir()` cada vez que `is_on_floor()` es verdadero y **no** tienen una variable `muriendo`, ¿qué pasa en el segundo frame en el piso? ¿Y qué pasa si "morir" tiene **Loop** activado?

Rta = Si no usola variable muriendo el juego se vuelve loco pues reiniciando la muerte y el sonido en cada frame que el pájaro toque el suelo y si le dejo el loop activado el pájaro se queda muerto repitiendo la animación por siempre y nunca vuelve a aparecer arriba

4. ¿AudioStreamPlayer2D o AudioStreamPlayer para el golpe contra el piso? Justifiquen.

Rta = Uso un AudioStreamPlayer normal porque el golpe contra el suelo pasa siempre en el mismo sitio de la pantalla

y la cámara no se mueve siguiendo al pajaro

Guía para la 1. Con otros números: si una animación debe ir a 3× con vy = −600 y a 1× con vy = 0, una fórmula es 1 + (−vy) ÷ 300. Comprueben los dos extremos con su fórmula.

Guía para la 2. Con otros números: 5 cuadros a 20 FPS duran 5 ÷ 20 = 0.25 s, que son 0.25 × 60 = 15 frames.

Guía para la 3. `play("morir")` sobre la animación que ya está sonando no la reinicia, pero si cada frame hacen todo `morir()` desde el principio (sonido incluido), ¿cuántos golpes suenan? ¿Y cuándo "termina" una canción que está en repetición?

Guía para la 4. ¿El golpe sale de un lugar del mundo, o es un aviso del juego? En este juego la cámara no sigue al pájaro en Y: ¿cambia algo eso?

Programar

1. En `actualizar_animacion()`, ajusten `speed_scale` con su fórmula cuando sube, y déjenlo en 1 cuando planea.

2. En el panel SpriteFrames, creen la animación **morir** con los cuadros **5 a 10** de la hoja (Add frames from Sprite Sheet: 11 horizontal, 1 vertical), a **10 FPS** y con **Loop desactivado**.
3. Agreguen un **AudioStreamPlayer** llamado **Golpe** con *assets/golpe.wav*.
4. Escriban `morir()`: mientras dura la animación, el pájaro no se mueve; al terminar, vuelve a su posición inicial, quieto.
5. En `_physics_process`, si el pájaro toca el piso (`is_on_floor()`), que muera.
6. Activen un momento el Loop de "morir" y describan qué pasa.

Esqueleto. Escriban el código debajo de cada comentario:

```
var inicio = Vector2.ZERO   # guárdenlo en _ready()
var muriendo = false


func morir():
    # 1. Marquen que está muriendo y detengan al pájaro (velocity).
    # 2. speed_scale en 1 y rotation en 0; reproduzcan "morir" y el golpe.
    # 3. Esperen a que la animación termine (await ...).
    # 4. Vuelvan al inicio, quietos, con "planear", y dejen de estar muriendo.
```

Pistas.

- La primera línea de `_physics_process` puede ser `if muriendo: return`: así nada se mueve mientras muere.
- Después de `mover()`: `if is_on_floor(): morir()` y `return`.
- La posición inicial se guarda en `_ready()`: `inicio = position`.

Cómo saber que funciona. Justo después del aleteo `speed_scale` vale 2; 9 frames después, **1.58**; planeando, 1. Al tocar el piso, el pájaro se aplasta durante **0.6 s (36 frames)**, suena el golpe **una vez** y reaparece en **(160, 320)**. Con Loop activado se queda muriendo para siempre.

Algo del codigo quedo asi:

Las lineas del _physics_process

if muriendo:

return

mover(delta)

```
if is_on_floor():

morir()

return



actualizar_animacion()

inclinar()
```

Los cambios de actualizar animacion:

```
func actualizar_animacion():

var vy = get_real_velocity().y

print(sprite.animation)

if vy < -1:                # sube

sprite.play("aletear")

sprite.speed_scale = 1.0 + (-vy) / 350.0

else:

sprite.play("planear")

sprite.speed_scale = 1.0
```

Y la nueva funcion que es la de morir:

```
func morir():

muriendo = true

velocity = Vector2.ZERO

sprite.speed_scale = 1.0

sprite.rotation = 0.0
```

sprite.play("morir")

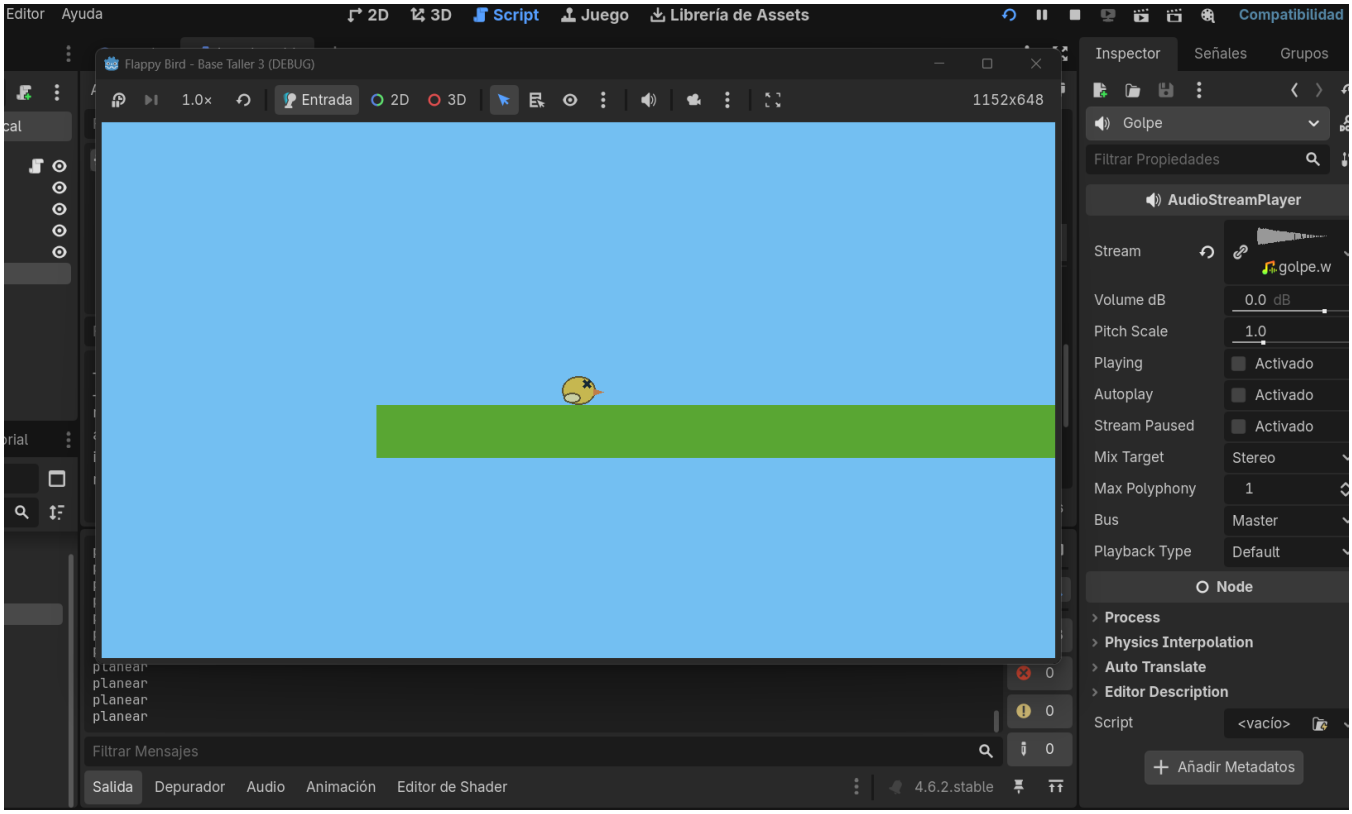
sonido_golpe.play()

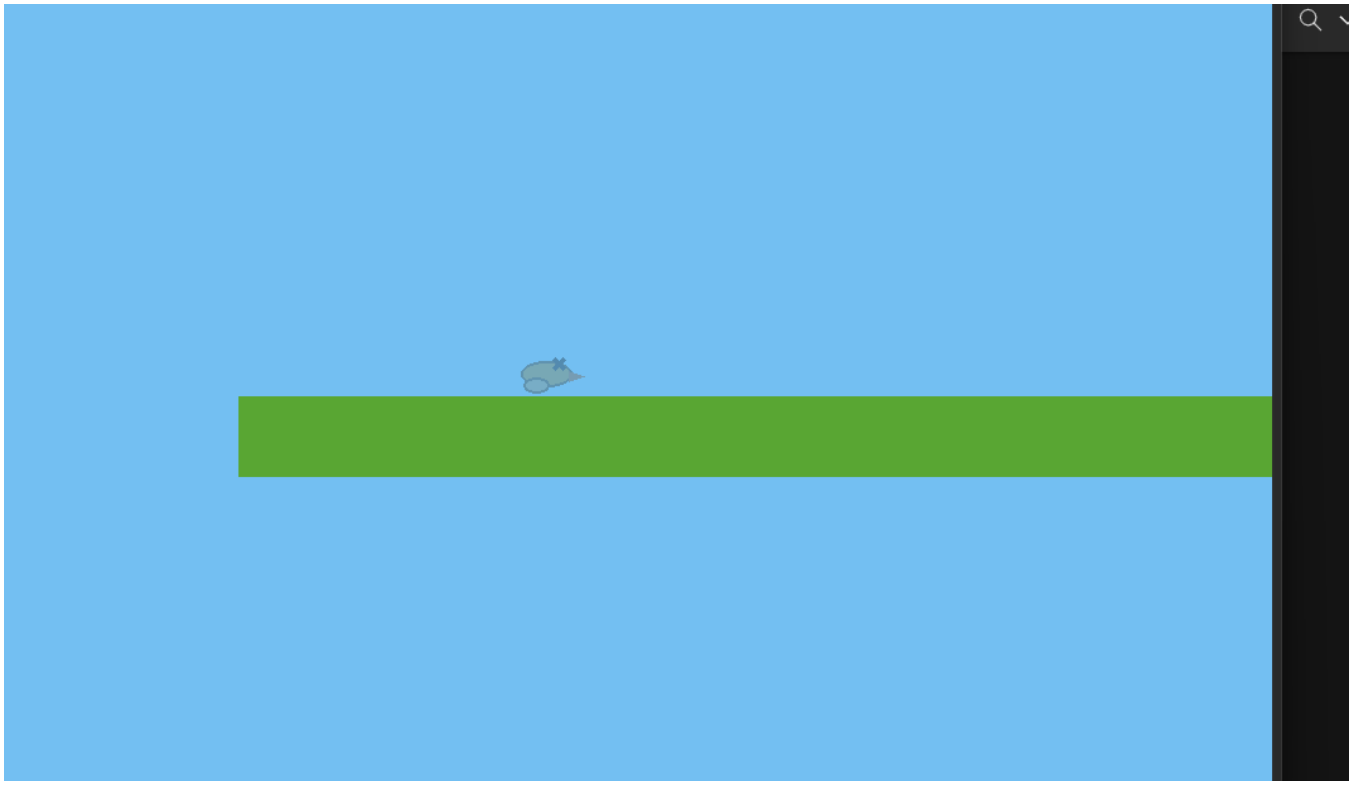
await sprite.animation_finished

global_position = inicio

sprite.play("planear")

muriendo = false





Mi predicción funciona ya que al tocar el piso el pajaro se frena suena el golpe una sola vez mientras dura la animación de la muerte en el tiempo estimado y luego reaparece arriba en la posición inicial listo para seguir juganod

Extra — Para quien termine todo

1. Que el aleteo suene más agudo cuanto más rápido venía cayendo (`pitch_scale`).
2. Que la inclinación no salte de golpe a −25° al aletear, sino que gire suave hacia ese ángulo (`lerp_angle`).

Documento de entrega

1. Nombre completo.
2. Por cada nivel: las respuestas de **Antes de programar**, el código, las capturas y una línea sobre si la predicción coincidió y, si no, por qué.
3. Respuestas cortas, con sus palabras:

4. **a.** ¿Por qué la animación se decide con la velocidad real y no con la tecla?
5. **b.** ¿Por qué aquí el sonido se reinicia en cada aleteo, y en Pac-Man el waka no?
6. **c.** ¿Qué pasa sin la variable `muriendo`, y qué pasa con "morir" en Loop?

Rta = Sin la variable muriendo el juego se bloquea repitiendo el choque en cada frame y con loop el pájaro se queda muerto para siempre y no reaparece

7. **d.** ¿Qué reproductor usaron para el golpe, y por qué?

Rta = Se usa un AudioStreamPlayer normal porque la pantalla no se mueve verticalmente y el golpe siempre suena en el mismo lugar

8. **Anexo IA:** si usaron IA, copien cada prompt que escribieron y la respuesta completa que obtuvieron. Si no la usaron, escriban "No se usó IA". Si usaron IA y no hay anexo, el taller no se califica.

La use especificamente para entender como hacer y aplicar la formula del nivel 3:

PROMT:

Oye una pregunta tengo una duda de una sola cosa simple pero como complicada o bueno es que la verdad no entendi y es en el nivel 3 en la parte que habla de la formula quisiera que me explicaras eso osea como hago esa formula o que debo seguir especificamente y quisiera que me explicaras detalladamente pa que es esa formula o si como la hago es que la verdad mejor dicho de esa formula osea ese punto no entendi el como hacerla o de donde la saco

RESPUESTA:

¡Hola! No te preocupes, ese punto del **Nivel 3** suele generar dudas porque combina matemáticas simples con programación, pero una vez que entiendes la lógica de qué busca hacer en el juego, verás que es muy sencillo de armar.

PDF

A continuación te explico detalladamente **qué es**, **para qué sirve** y **cómo se construye esa fórmula paso a paso** para que puedas completarlo en tu guía.

1. ¿Para qué es esa fórmula?

En el juego de *Flappy Bird*, cuando el personaje aletea, sale impulsado hacia arriba con mucha fuerza y sus alas deberían batirse súper rápido. A medida que pierde fuerza y llega al punto más alto de su salto (cuando su velocidad hacia arriba disminuye a 0), sus alas deberían batirse a una velocidad normal.

PDF+ 2

Para lograr este efecto en Godot, usamos la propiedad **`speed_scale`** de la animación (que es un multiplicador de velocidad: *1.01.0*1.0 es velocidad normal, *2.02.0*2.0 es el doble de rápido, etc.).

PDF

- **El objetivo de la fórmula:** Calcular dinámicamente el valor de `speed_scale` usando la velocidad vertical del pájaro (vy), de tal manera que:

- Justo al presionar aletear (cuando la velocidad `vy = -350`), el batido vaya al doble de rápido (*speed_scale=2.0|text{speed|_scale} = 2.0*speed_scale=2.0).

PDF+ 2

- En el punto más alto del salto (cuando la velocidad `vy = 0`), el batido baje a velocidad normal (*speed_scale=1.0|text{speed|_scale} = 1.0*speed_scale=1.0).

PDF

2. ¿De dónde sale y cómo se hace la fórmula?

Recordemos los datos que nos da el taller:

- **vy**: Es la velocidad vertical del personaje tomada con `get_real_velocity().y`. En Godot, hacia arriba los valores son **negativos** (por eso en el taller dice que la velocidad de impulso es *−350-350*−350).

PDF+ 4

- **`impulso`**: Vale *350350*350 (es el valor absoluto de la fuerza con la que sube).

PDF+ 1

En la **Guía para la 1** del Nivel 3, el profesor te da la pista exacta de la estructura de la fórmula:

PDF
*Fo´rmula general=1+−vyimpulso|text{Fórmula general} = 1 + |frac{-vy}{|text{impulso}}*Fo´rmula general=1+impulso−vy

¿Por qué funciona así?

1. **Cuando recién aleteas (*vy=−350vy = -350*vy=−350):**

*speed_scale=1+−(−350)350=1+350350=1+1=2.0|text{speed|_scale} = 1 + |frac{-(-350)}{350} = 1 + |frac{350}{350} = 1 + 1 = 2.0*speed_scale=1+350−(−350) =1+350350 =1+1=2.0

(¡Las alas van al doble de rápido!)

PDF+ 1

2. **Cuando llega al punto más alto (*vy=0vy = 0*vy=0):**

*speed_scale=1+0350=1+0=1.0|text{speed|_scale} = 1 + |frac{0}{350} = 1 + 0 = 1.0*speed_scale=1+3500 =1+0=1.0

(Las alas van a velocidad normal).

PDF

3. Respuestas específicas para los puntos del taller

Con esto claro, ya puedes responder las preguntas teóricas del taller y escribir el código:

A. Para la pregunta "Antes de programar #1":

Te preguntan cuánto vale la fórmula a los 9 frames del aleteo, cuando $vy \approx -203$vy ≈−203:

PDF

- **Ecuación/Fórmula:** `speed_scale = 1.0 + (-vy / 350.0)`

PDF

- **Cálculo para $vy = -203$vy=−203:**

$speed_scale = 1 + -(-203)350 = 1 + 203350 = 1 + 0.58 = 1.58$speed_scale=1+−(−203)350=1+203350=1+0.58=1.58

- **Respuesta que debes colocar:** En el frame 9, el valor de `speed_scale` es aproximadamente **$1.581.58$1.58**.

PDF

B. ¿Cómo colocarla en el código en Godot?

Dentro de tu función `actualizar_animacion()`, debes aplicar la fórmula únicamente **cuando el pájaro esté subiendo** (es decir, cuando $vy < -1$vy<−1). Cuando no esté subiendo (cuando planee o caiga), debes dejar `speed_scale` en $1$1.

PDF+ 2

Tu código dentro de `actualizar_animacion()` quedaría de la siguiente manera:

GDScript

```
func actualizar_animacion():
	var vy = get_real_velocity().y

	if vy < -1: # sube
		# Aplicamos la fórmula para acelerar el aleteo
		sprite.speed_scale = 1.0 + (-vy / 350.0)
		sprite.play("aletear")
	else:
		# Al planear o caer, vuelve a la velocidad normal
		sprite.speed_scale = 1.0
		sprite.play("planear")
```

Resumen para verificar que funciona

Cuando pruebes el juego en la consola/pantalla:

- Al presionar la tecla de aletear, verás que las alas aletean súper rápido por una fracción de segundo.

PDF+ 1

- Conforme el pájaro pierde impulso y llega a la cima de su curva de salto, las alas se van desacelerando progresivamente hasta que se queda planeando.

PDF

¿Te queda clara la lógica o necesitas que profundicemos en alguna otra parte del Nivel 3 (como lo de la variable `muriendo` o el `await`)?

SEGUNDO PROMT:

no me digas nada de codigo ni eso quiero es entender la formula porfa hazla de forma mas simple es que aun no la entendi del todo

RESPUESTA:

¡Claro que sí! Olvidémonos por completo del código y veámoslo como una simple receta o un truco lógico.

Imagina que quieres que la velocidad del aleteo de las alas funcione como una especie de "medidor" que cambia automáticamente dependiendo de qué tan fuerte va subiendo el pájaro.

La fórmula es simplemente una forma de hacer una **conversión** usando dos reglas fijas que te pide el profesor:

1. **Cuando el pájaro acaba de saltar con toda la fuerza** (es decir, cuando va subiendo a su máxima velocidad de -350-350–350), queremos que las alas se muevan al **doble de velocidad** (o sea, a $2$2).

PDF+ 1

2. **Cuando el pájaro llega arriba del todo y su velocidad de subida llega a cero** ($0$0), queremos que las alas se muevan a su **velocidad normal** (o sea, a $1$1).

PDF

Entonces, ¿cómo funciona paso a paso la fórmula `$1 + (-vy / 350)$`?

Piénsalo como una balanza o una regla de tres oculta:

- El número **1** que está al principio es tu **base normal**: te asegura que la velocidad de la animación nunca va a ser menor a $1$1 (es decir, nunca va a ir en cámara lenta).
- La parte de **`-vy / 350`** es el **extra de velocidad** que se le suma a ese $1$1 solo cuando el pájaro va hacia arriba:
    - Como la velocidad al subir es negativa (por ejemplo, -350-350–350), le ponemos un signo menos adelante (`-vy`) para volverla positiva y poder hacer la cuenta fácil.

PDF+ 1

    - Si divides esa velocidad actual entre el impulso total ($350$350350), obtienes un porcentaje de cuánta fuerza le queda al salto.

PDF+ 1

    -
        - Ejemplo: Si vas a la mitad de la fuerza de subida ($175$175175), la división te da $0.5$0.5. Se lo sumas al $1$1 base y te queda $1.5$1.5 (las alas se mueven un 50% más rápido de lo normal).
    -

- Si ya llegaste arriba y la velocidad es $00$0, la división da $00$0. Se lo sumas al $11$1 base y te queda exactamente $11$1 (velocidad normal).

PDF

En pocas palabras: **es una formulita matemática para ir frenando el movimiento de las alas de forma gradual**, empezando muy rápido justo después del salto y volviendo a la normalidad conforme el pájaro va perdiendo impulso hacia arriba.

PDF

¿Tiene más sentido viéndolo así como un porcentaje de fuerza que va disminuyendo?

Cómo se califica (0.0 a 5.0)

Cada franja exige todo lo de la anterior. Si una franja queda a medias, la nota queda entre las dos (por ejemplo, 3.5 o 4.5).

Nota	Grupos de 3 horas	Grupos de 2 horas
3.0	Nivel 1 funcionando: animación decidida con la velocidad real, frames de "aletear" contados y comparados con la predicción 2.	Nivel 1 funcionando: animación decidida con la velocidad real.
4.0	Nivel 2: inclinación con `clampf` y `deg_to_rad`, comprobada con las predicciones 1 y 2, y aleteo que suena cada vez, comparado con la versión que pregunta `playing`. Respuestas a y b.	Nivel 2: inclinación comprobada con la predicción 1, y aleteo que suena cada vez. Respuestas a y b.
5.0	Nivel 3 completo: `speed_scale` con fórmula comprobada, muerte con `await` que bloquea el movimiento y vuelve al inicio, `muriendo` y Loop explicados, y reproductor justificado. Respuestas c y d.	Nivel 3 sin `speed_scale`: muerte con `await` que bloquea el movimiento y vuelve al inicio, y Loop explicado. Respuestas c y d.

Hacer bien lo pedido hasta el nivel 2 da 4.0. El 5.0 es para quien resuelve y explica el nivel 3.